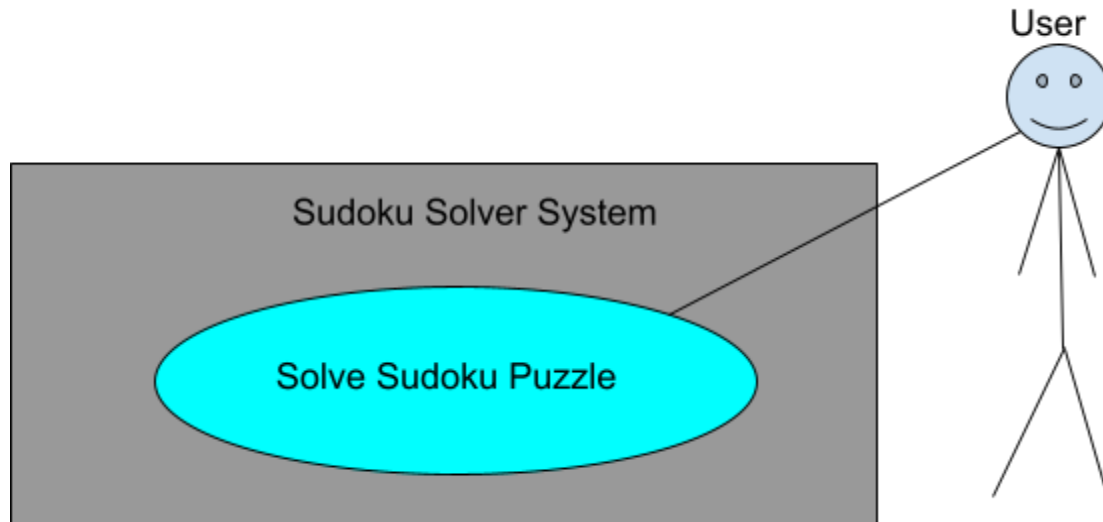


Assignment 4 Use Case Diagram

Cesar Olono, EECS 348



1. Use case identified

Solve Sudoku Puzzle

2. Brief description

The user runs the program. It reads each Sudoku puzzle file, solves the puzzle with a recursive search, and prints the original puzzle and every solution, or "No solution found."

3. Step-by-step flow

1. The user runs the program.
2. The program opens puzzle1.txt.
3. The program reads the puzzle into a 9x9 grid.
4. The program prints the file name and the original puzzle.
5. The program checks that the given digits do not break any Sudoku rule.
6. The program searches for solutions, trying digits in empty cells and backing up when stuck.
7. The program prints each solution as it finds one.
8. If no solution was found, the program prints "No solution found."
9. The program repeats steps 2 to 8 for puzzle2.txt through puzzle5.txt.

4. Detailed description

Use case name: Solve Sudoku Puzzle

Actor: User

Goal: Get every solution for each of the five puzzle files.

Preconditions:

- puzzle1.txt through puzzle5.txt are in the same folder as the program.
- Each file has 81 cells, using digits 1 to 9 and "_" for blanks.

Main flow: Steps 1 to 9 above.

Alternate flows:

- A file cannot be opened or is not a valid puzzle (wrong characters or not 81 cells). The program prints an error message for that file and moves on to the next file.
- The given digits already break a rule (for example, two 5s in one row). The program finds no solution and prints "No solution found."
- The puzzle has more than one solution. The program keeps searching and prints all of them.

Postconditions:

- For every file, the screen shows the file name, the original puzzle, and either every solution or "No solution found."

Notes: The search always picks the empty cell with the fewest legal digits. Claude helped me write this use case.